

Sesión Presentación

22 Abril 2024

- 1. Presentación**
- 2. Overview Modulo**
- 3. Sistemas Big data**
- 4. Apache Kafka**

1/ Presentación



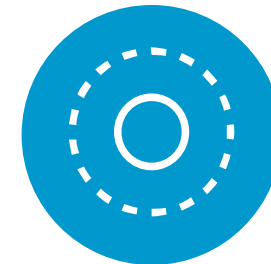
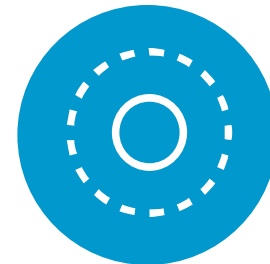
David Carvajal Head Data & IA Data and IA Lover

Experiencia....MUUUUUUCHAAAA

1/ Overview Módulo

UF1 Aplicación de técnicas de integración, procesamiento y análisis de información.

L02. Técnicas y procesos de extracción de la información de los datos .

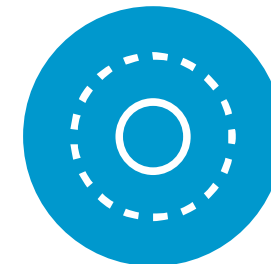
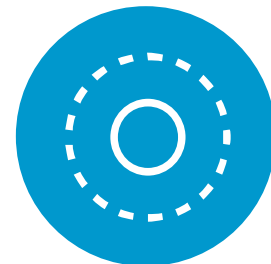


UF2 Técnicas de representación y minería de datos

L03. Métodos y algoritmos de minería de datos

UF 3 Sistemas de almacenamiento

Sistemas de gestión almacenamiento HDFS.
Apache Kafka, Motores y bbdd NoSQL



UF 4 Conceptos y visualizacion

L04 Herramientas de visualización de datos

3 /UF1 Aplicación de técnicas de integración, procesamiento y análisis de información

Antes que comencemos cualquier discusión acerca del tema que nos ocupa en esta lección, es sumamente importante que aclaremos la diferencia entre Datos, Información y Conocimiento.

Esta distinción es clave para entender cómo los sistemas de procesamiento de datos, como los involucrados en Big Data, transforman los insumos crudos en resultados útiles para la toma de decisiones.



Diferencias clave:

Concepto	Definición	Función	Ejemplo
Datos	Elementos crudos y sin procesar, sin significado propio.	Recoger observaciones.	"102.3°F", "3000 unidades", "Rojo".
Información	Datos organizados y estructurados con contexto, que ofrecen significado.	Proporcionar contexto.	"La temperatura actual es 102.3°F en la ciudad X".
Conocimiento	Interpretación de la información, aplicada para tomar decisiones y resolver problemas.	Entender y predecir patrones.	"Las ventas aumentan en verano por mayor demanda".

Relación entre los tres:

- Datos → Procesamiento → Información → Análisis → Conocimiento

La progresión desde los datos hasta el conocimiento muestra cómo los sistemas de análisis y Big Data permiten transformar grandes volúmenes de datos aparentemente desorganizados en información relevante y, finalmente, en conocimiento útil para la toma de decisiones empresariales o el entendimiento de fenómenos complejos.

Los **datos** son los elementos más básicos y primarios. Se componen de hechos y cifras sin un contexto específico o interpretación asociada. Los datos por sí solos no tienen un significado intrínseco; son fragmentos de realidad observada, sin procesar ni analizar.

- **Características:**
 - Son simples valores o medidas.
 - No tienen contexto ni propósito inherente.
 - No han sido interpretados ni analizados.
- **Ejemplos:**
 - Números como: 25, 30, 85.
 - Valores de sensores: 102.3° F.
 - Texto simple: “Rojo”, “Azul”, “Lluvia”.



Los datos son el **insumo bruto** en muchos sistemas, pero no proporcionan conocimiento útil hasta que se procesan

La **información** surge cuando los datos se organizan y procesan de una manera que les da significado. A partir de los datos, la información es el siguiente nivel, donde se puede proporcionar un contexto y se puede entender cómo los datos se relacionan entre sí. La información responde a preguntas básicas como "qué", "quién", "dónde" y "cuándo".

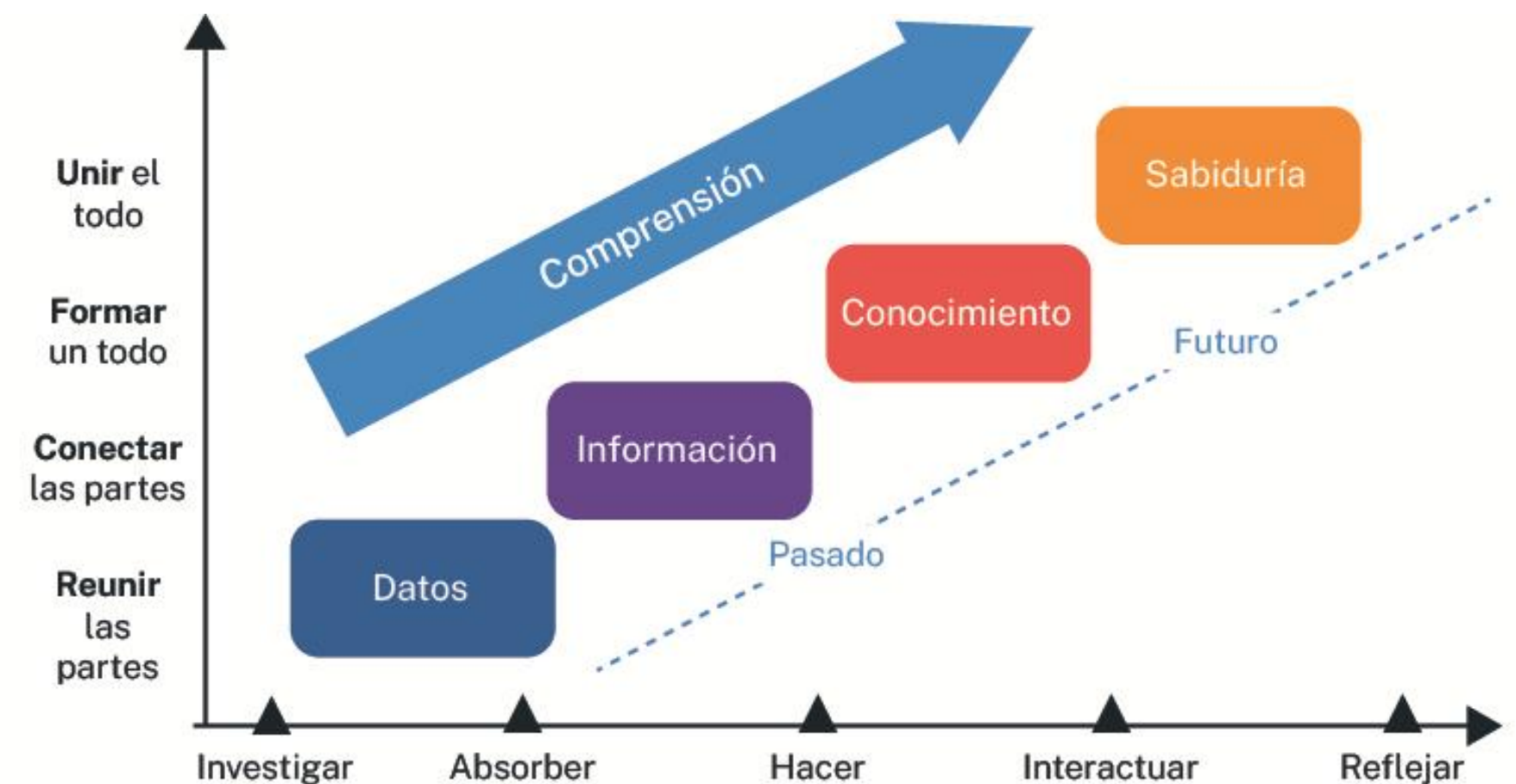
- **Características:**

- Los datos se organizan, clasifican o estructuran.
- Se le agrega contexto o propósito a los datos.
- La información es comprensible y útil para la toma de decisiones.

- **Ejemplos:**

- "La temperatura actual en Nueva York es 25 °C."
- "El número de ventas de septiembre fue de 5,000 unidades."
- "El tráfico en la autopista A1 es fluido."

La **información** es más útil que los datos porque ya ha sido procesada o interpretada para proporcionar relevancia. Por ejemplo, un conjunto de números de temperatura de sensores se convierte en información cuando se sabe a qué ciudad pertenecen y en qué día se registraron.



El **conocimiento** se obtiene cuando se interpreta la información y se conecta con la experiencia o el entendimiento previo. Implica el uso de la información para sacar conclusiones, entender patrones, identificar tendencias o tomar decisiones. El conocimiento responde a preguntas más profundas, como "por qué" y "cómo".

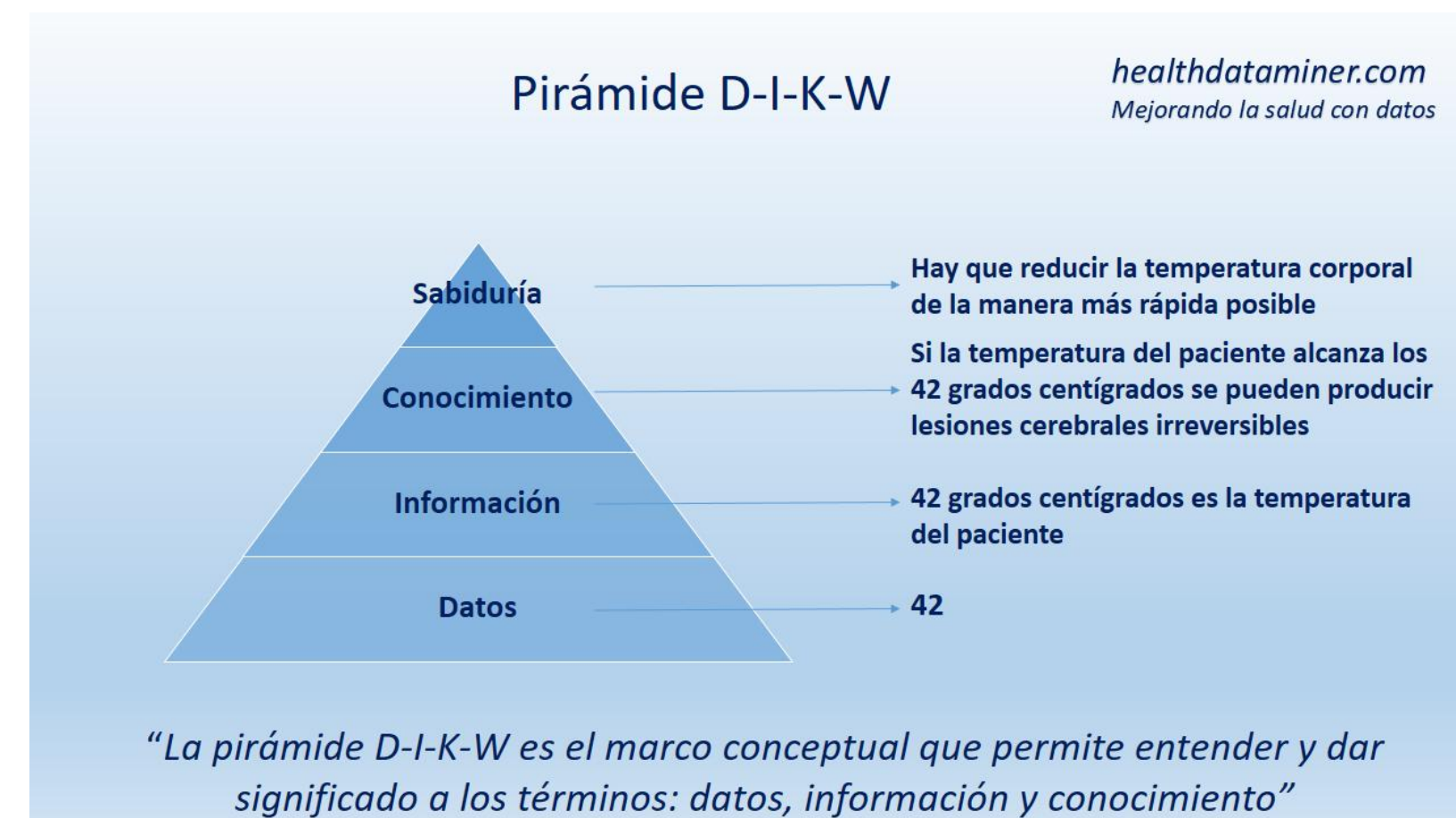
- **Características:**

- Es la interpretación o comprensión de la información.
- Requiere análisis, contexto adicional y experiencia.
- Se utiliza para tomar decisiones o para resolver problemas.

- **Ejemplos:**

- "Sabemos que las ventas aumentan durante los meses de vacaciones debido a una mayor demanda."
- "Dado que la temperatura es baja y hay previsión de lluvia, es probable que el tráfico empeore en la tarde."
- "Los patrones de compra sugieren que los consumidores prefieren el producto A en los meses de verano."

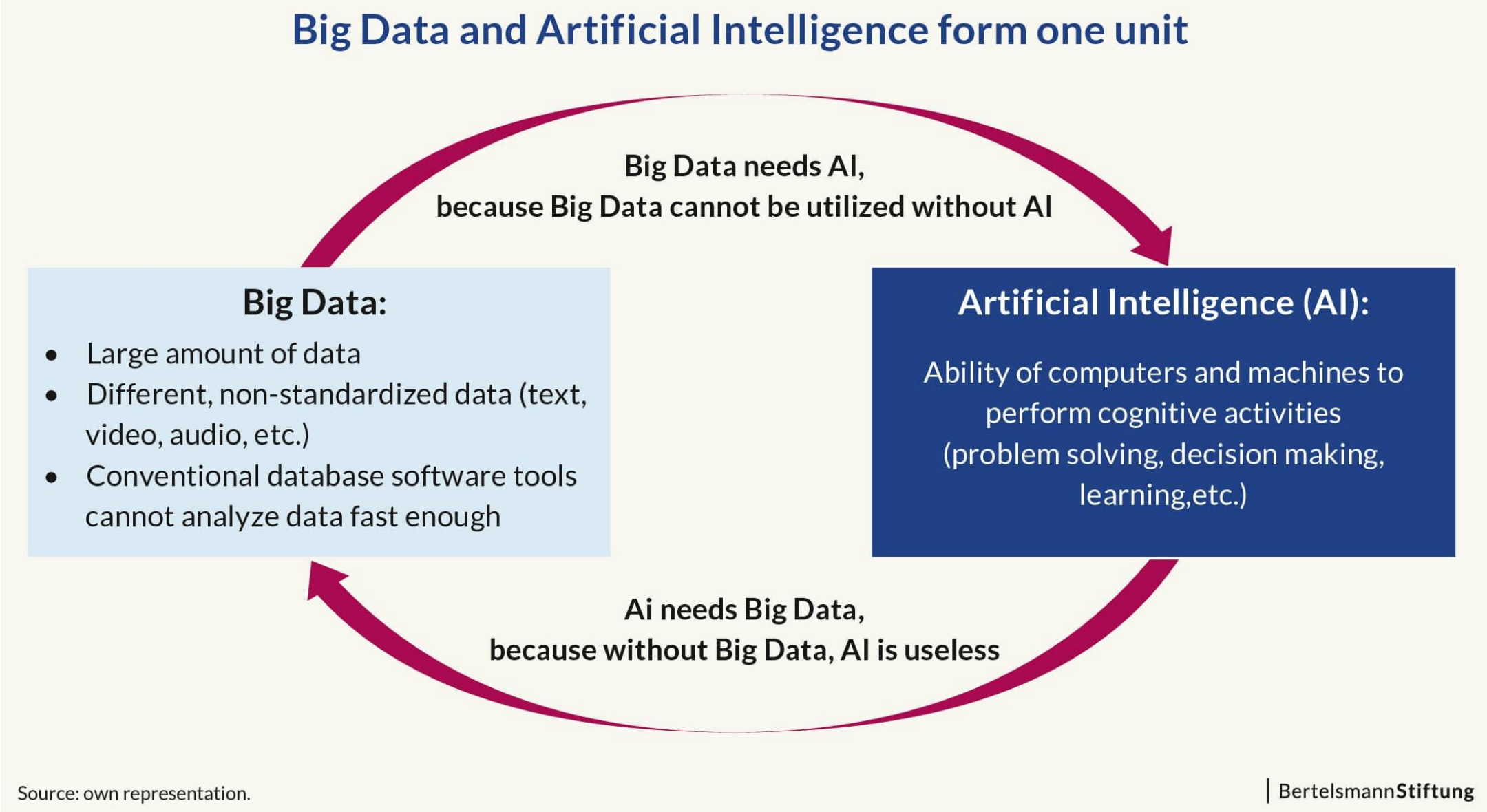
El **conocimiento** permite actuar de manera efectiva sobre la base de la información. Mientras que la información te dice lo que está sucediendo, el conocimiento te dice **por qué** sucede y **cómo** podrías influir en ello.



**¿Qué sucede en
Un MINUTO en
INTERNET?**

Un estudio de McKinsey Global Institute reveló que las empresas que utilizan estas herramientas son hasta un **23% más propensas a obtener una ventaja**

FIGURE 1: Stylized relationship between Big Data and artificial intelligence (AI).



Big Data en este contexto:

En el mundo del Big Data, uno de los grandes desafíos es precisamente la conversión efectiva de datos (que pueden ser masivos, heterogéneos y desestructurados) en información significativa y luego usar esa información para generar conocimiento que aporte valor a la organización o la sociedad.

Las 5 V del Big Data:

1. **Volumen.** Las organizaciones tienen que lidiar con miles de Megabytes y Terabytes. Estamos en la era de los Exa y los Zetabytes
2. **Velocidad.** : La velocidad a la que se actualizan nuestros datos provoca que, si no lo manejamos adecuadamente, cuando generemos algún conocimiento, será erróneo, pues los datos de origen ya cambiaron.
3. **Variedad.** Esta cualidad se refiere a la cantidad de formatos y tipos de datos que las organizaciones ahora tienen que manejar.
4. **Valor:** El valor puede definirse como la utilidad de los datos para la empresa. Esto quiere decir que podemos procesar los datos en volúmenes increíbles y además lo hacemos a altas velocidades, pero de nada vale si no es de valor para la empresa. Para mí este es un elemento vital.
5. **Veracidad:** Este tipo de cualidad se refiere a que los datos que se procesan con estas tecnologías tienen que ser de calidad.



Fig.10. Las 5 V del Big Data.

El ciclo de vida del dato y la arquitectura Big Data están íntimamente relacionados, ya que ambos describen el recorrido que siguen los datos, desde su creación hasta su eliminación o transformación en conocimiento útil. Cada fase del ciclo de vida del dato está respaldada por una o varias capas de la arquitectura Big Data, facilitando la administración y el procesamiento a gran escala.



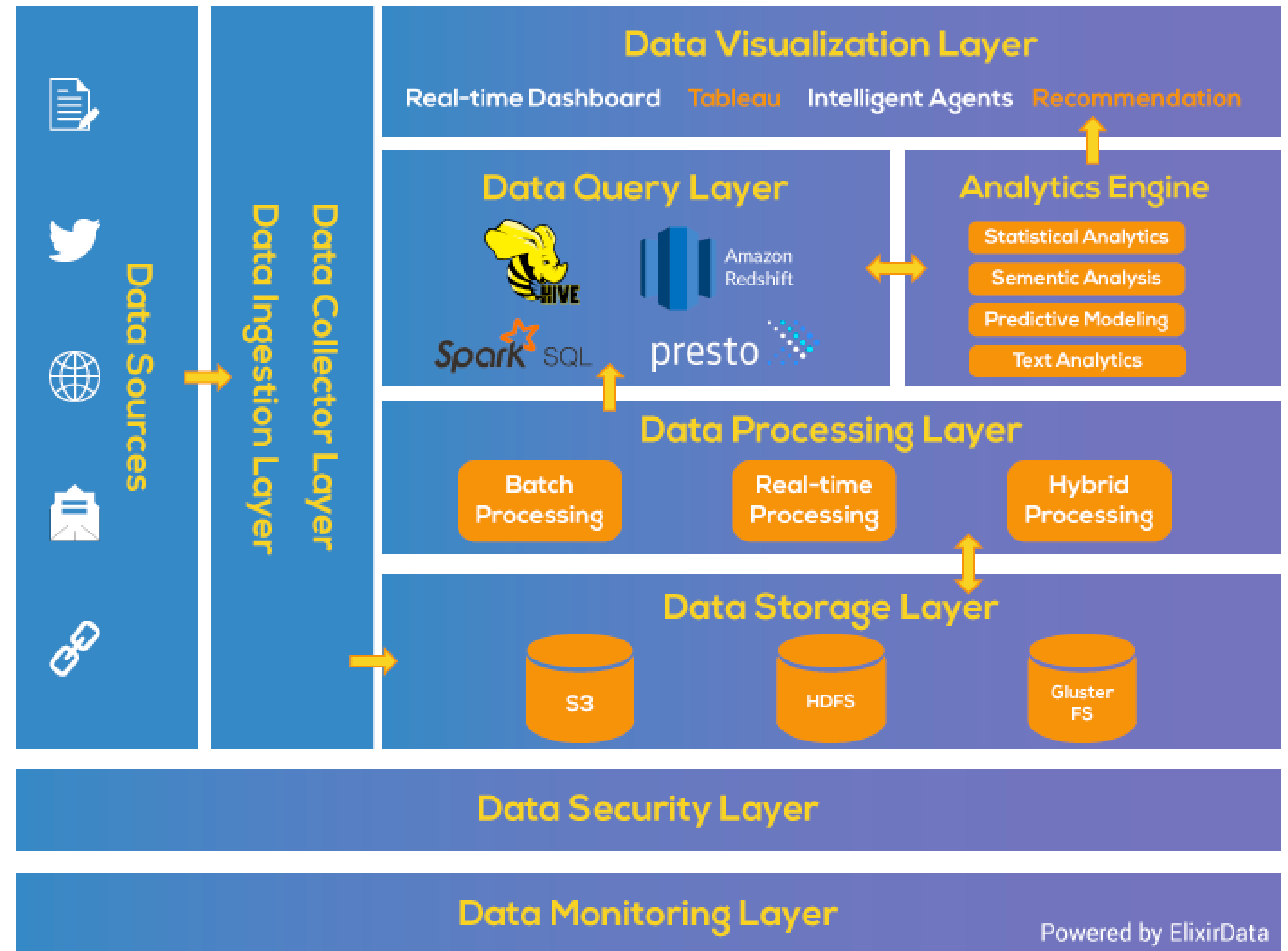
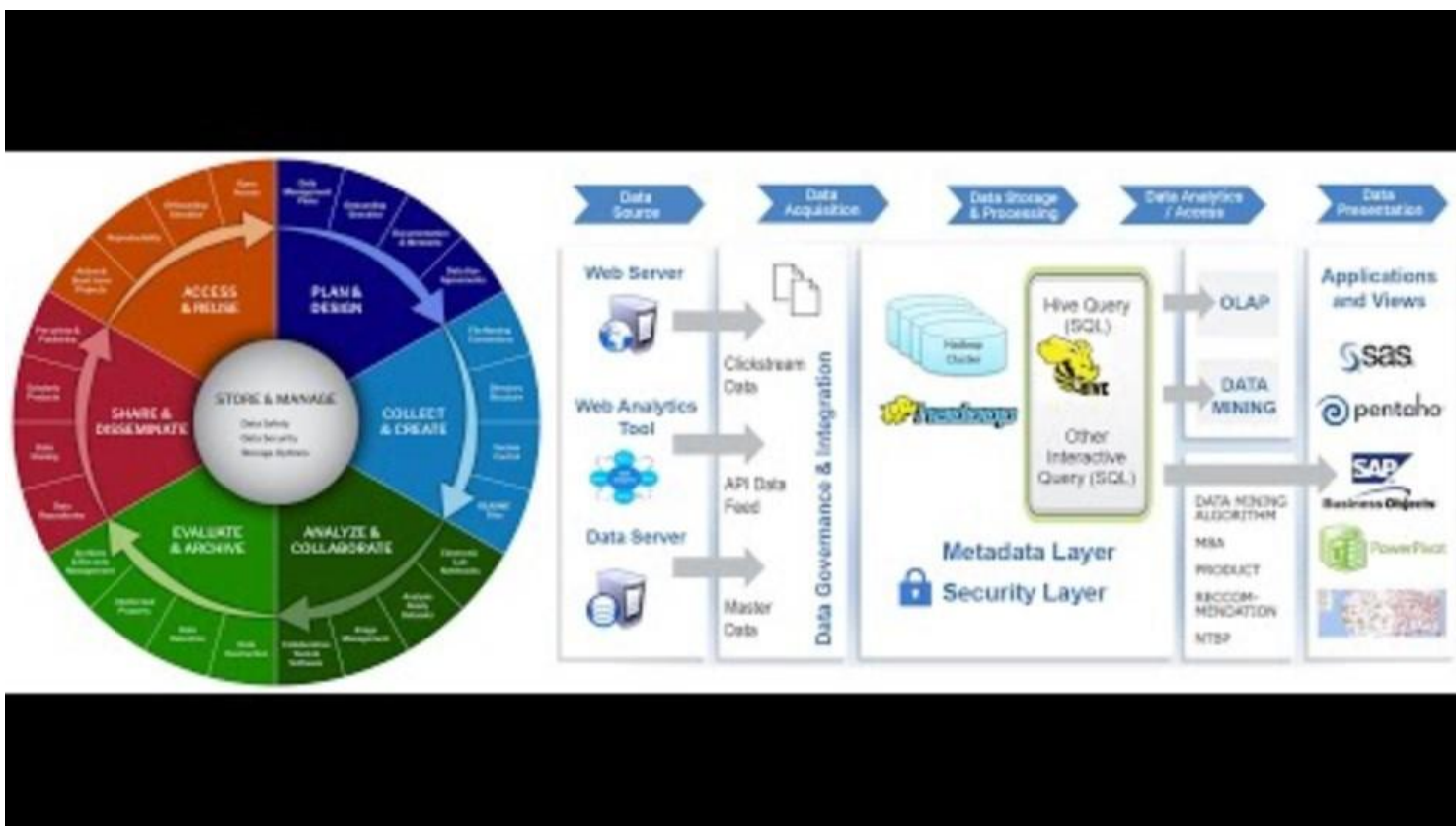
Fases del ciclo de vida del dato:

1. Generación y adquisición de datos
2. Almacenamiento de datos
3. Procesamiento de datos
4. Análisis y modelado de datos
5. Visualización y explotación de los datos
6. Gobernanza y seguridad de los datos
7. Retención o eliminación de los datos

Arquitectura Big Data: Definición y Componentes

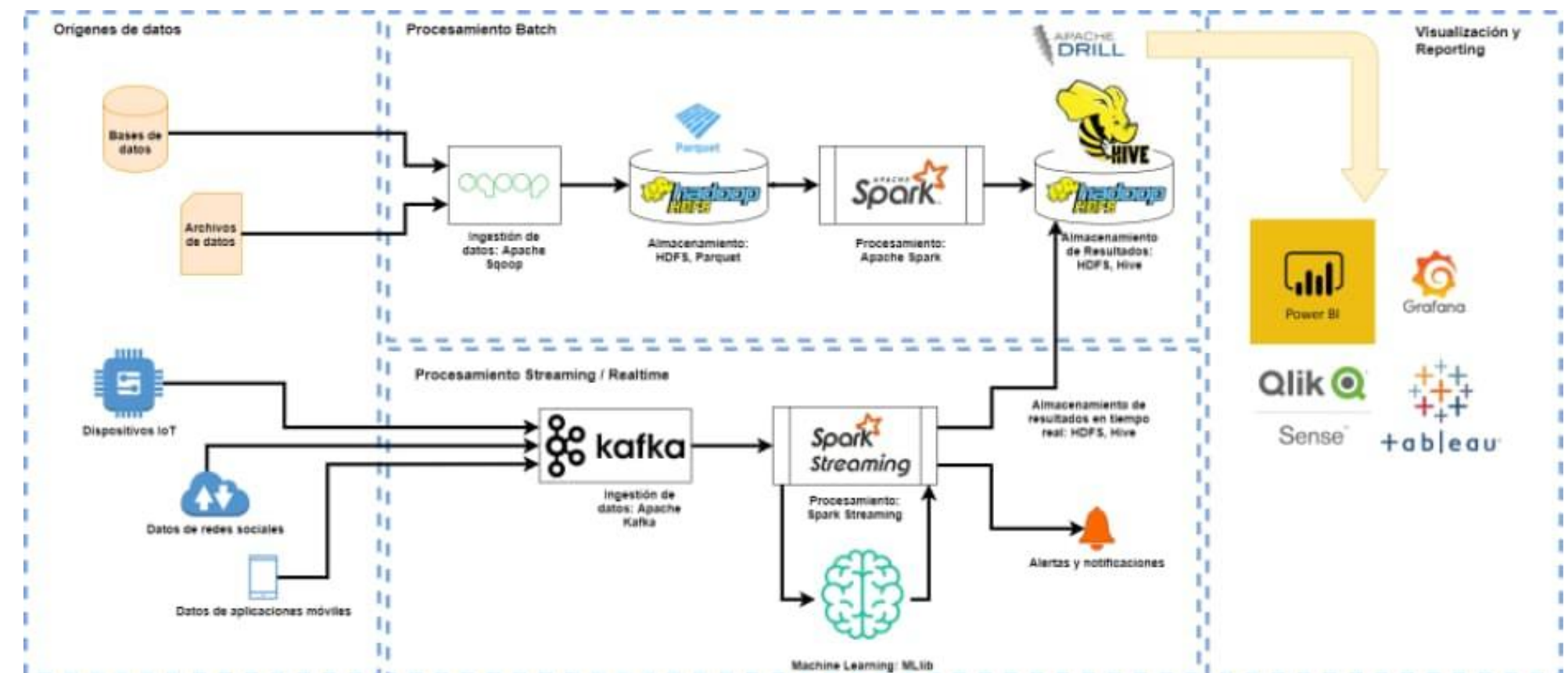
Una arquitectura Big Data es el diseño conceptual y la estructura que define cómo se recopilan, almacenan, procesan y analizan grandes volúmenes de datos en una organización.

Está compuesta por varias **capas** y **componentes**, cada una con una función específica, trabajando juntas para manejar la complejidad del procesamiento de datos masivos y heterogéneos



Hadoop es un framework de software de código abierto que permite almacenar y procesar grandes conjuntos de datos de forma distribuida y paralela en clústeres de computadoras. Es una herramienta fundamental en el mundo del Big Data debido a su escalabilidad y capacidad para manejar datos estructurados y no estructurados.

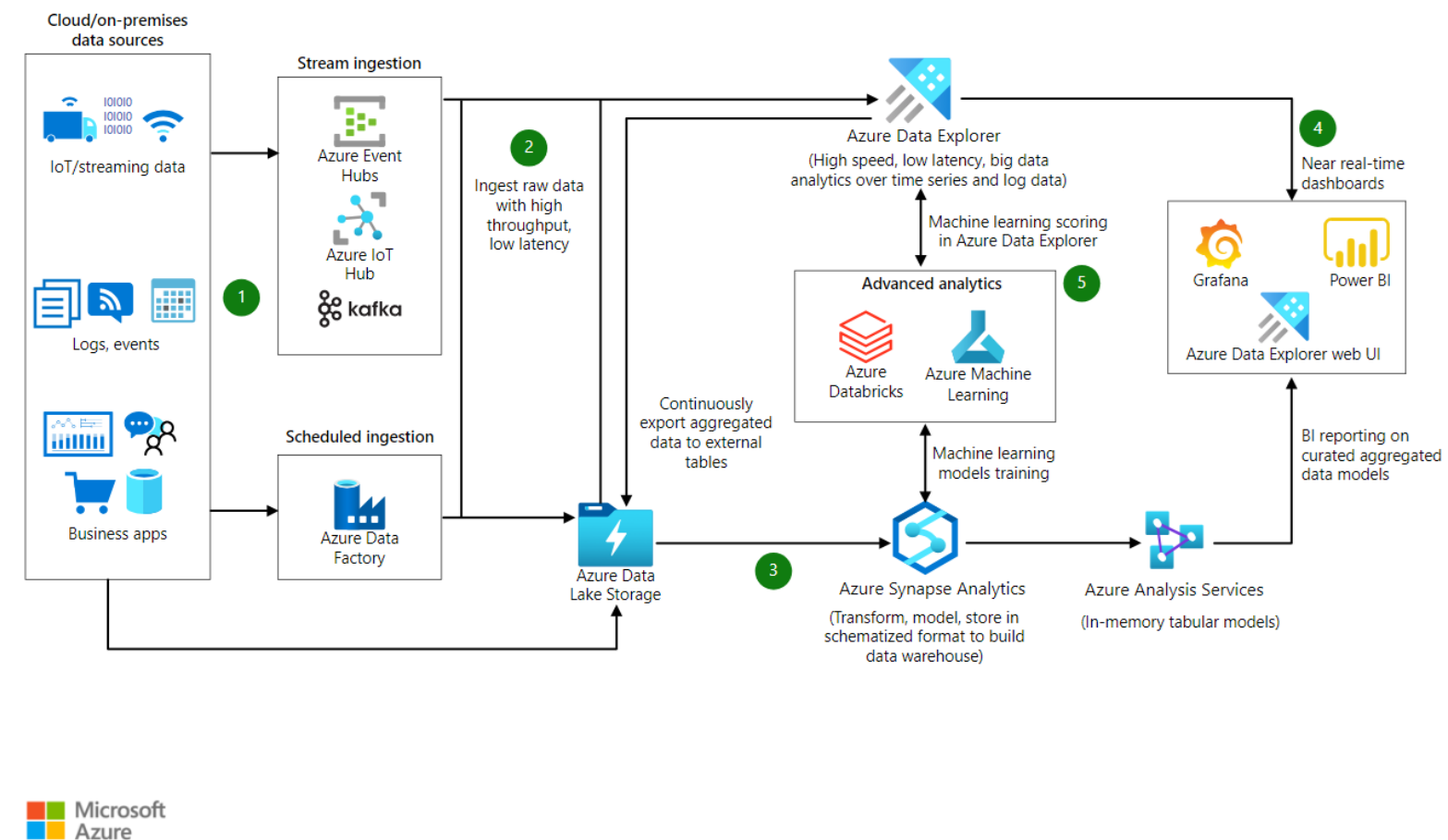
- **Escalabilidad:** Capacidad de manejar grandes volúmenes de datos.
- **Costo-efectividad:** Utiliza hardware commodity para construir clústeres.
- **Tolerancia a fallos:** Replicación de datos para garantizar la disponibilidad.
- **Flexibilidad:** Soporta una amplia variedad de formatos de datos.
- **Comunidad activa:** Gran comunidad de usuarios y desarrolladores.
- **Casos de Uso Típicos:**
 - **Análisis de grandes volúmenes de datos:** Log files, datos de sensores, redes sociales, etc.
 - **Almacenamiento de datos históricos:** Archivos de respaldo, registros de transacciones.
 - **Procesamiento de datos en tiempo real:** Análisis de streaming de datos.
 - **Machine Learning:** Entrenamiento de modelos de machine learning en grandes conjuntos de datos.



Azure proporciona una plataforma completa para gestionar y analizar grandes conjuntos de datos, desde la ingesta hasta la visualización. A diferencia de GCP, Azure ofrece una mayor flexibilidad al permitir la integración de tecnologías open source con servicios nativos..

Componentes Clave:

- **Azure Data Lake Storage Gen2:** Un sistema de archivos escalable y duradero para almacenar datos sin estructurados de cualquier tamaño.
- **Azure Synapse Analytics:** Un servicio unificado que combina un almacén d datos, un área de trabajo de Apache Spark y un servicio de integración de datos. Permite realizar consultas SQL, ejecutar trabajos de Spark y construir pipelines de ETL.
- **Azure HDInsight:** Un servicio administrado que permite ejecutar clústeres de Hadoop, Spark, Hive, HBase y Kafka en Azure. Ideal para cargas de trabajo de procesamiento de datos a gran escala.
- **Azure Stream Analytics:** Un servicio de procesamiento de streaming de datos en tiempo real basado en consultas SQL. Perfecto para aplicaciones de IoT y análisis de flujos de datos.
- **Azure Databricks:** Una plataforma de análisis basada en Apache Spark optimizada para Azure. Ofrece una experiencia de usuario colaborativa y un rendimiento excepcional.

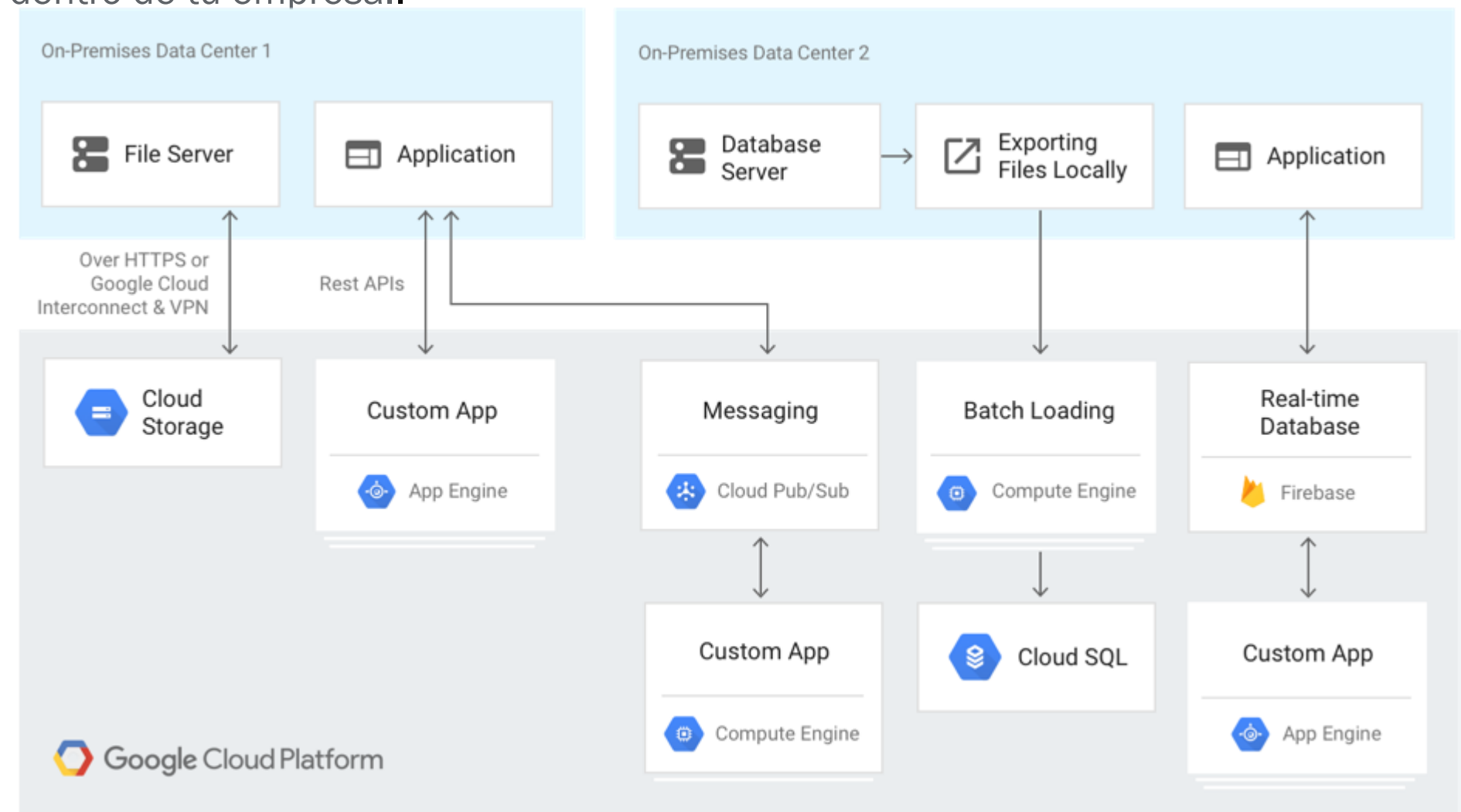


Inteligencia artificial para el análisis de datos: conclusiones y soluciones | Microsoft Azure

La nube de datos de Google proporciona una base de analíticas de datos unificada basada en BigQuery que reúne tus datos en un solo lugar, integrando datos estructurados y no estructurados con la IA para ofrecer información valiosa rápidamente en todo tu conjunto de datos. También puedes unificar tus datos operativos y analíticos de AlloyDB para PostgreSQL, Spanner y Cloud SQL con la federación de BigQuery sin mover ni copiar tus datos. La plataforma de datos unificada de Google te permite gestionar todo el ciclo de vida de sus datos y ayuda a simplificar los procesos de seguridad y gobernanza para diferentes tipos de usuarios dentro de tu empresa..

Componentes Clave

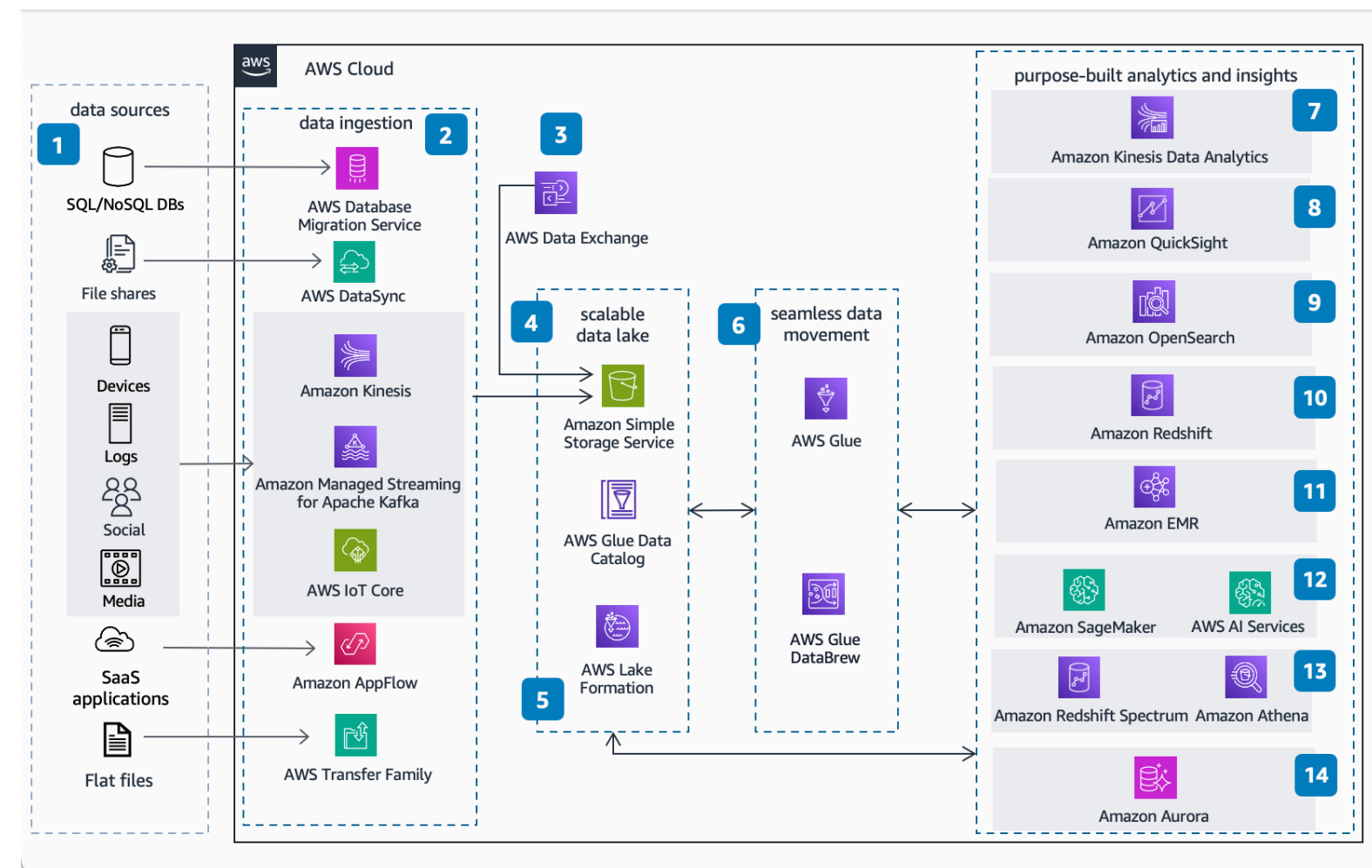
- Google Cloud Storage (GCS): Almacén de objetos escalable y duradero para almacenar grandes cantidades de datos sin estructura. Ideal para almacenar datos de respaldo, logs, imágenes, videos y otros tipos de archivos.
- Google Cloud Dataproc: Servicio administrado que permite ejecutar clústeres de Apache Spark y Hadoop en GCP. Facilita el procesamiento de datos a gran escala y la ejecución de trabajos de análisis de datos.
- BigQuery: Un almacén de datos totalmente administrado y sin servidor para análisis de datos a gran escala. Permite ejecutar consultas SQL complejas en petabytes de datos.
- Cloud Dataflow: Servicio de procesamiento de datos totalmente administrado para construir pipelines de procesamiento de datos a gran escala. Ideal para aplicaciones de procesamiento de datos en tiempo real y por lotes.
- Pub/Sub: Servicio de mensajería en la nube totalmente administrado para enviar y recibir mensajes entre aplicaciones de manera asíncrona. Perfecto para construir aplicaciones de streaming de datos y microservicios.



AWS ofrece una amplia gama de servicios para construir arquitecturas de Big Data robustas y escalables. AWS ofrece un conjunto completo de servicios de análisis que se adaptan a todas las necesidades de análisis de datos y permite a las organizaciones de todos los tamaños y sectores reinventar su negocio con datos. Desde almacenamiento y administración, gobernanza de datos, acciones y experiencias, AWS ofrece servicios especialmente diseñados que proporcionan la mejor relación precio-rendimiento, la mejor escalabilidad y el menor costo.

Componentes Clave

- **Amazon S3:** Un almacenamiento de objetos escalable y duradero para almacenar grandes volúmenes de datos sin estructura. Es ideal para almacenar datos de respaldo, logs, imágenes, videos y otros tipos de archivos.
- **Amazon EMR:** Un servicio administrado que permite ejecutar clústeres de Hadoop, Spark, Hive y otros frameworks de procesamiento de datos. Es perfecto para cargas de trabajo de procesamiento de datos a gran escala.
- **Amazon Redshift:** Un almacén de datos totalmente administrado y optimizado para consultas SQL analíticas. Ideal para análisis de datos a gran escala y creación de informes.
- **Amazon Athena:** Un servicio de análisis interactivo que permite ejecutar consultas SQL directamente sobre datos almacenados en S3. No requiere gestión de infraestructura.
- **Amazon Kinesis Data Streams:** Un servicio de streaming de datos en tiempo real que permite procesar grandes volúmenes de datos de manera continua.
- **Amazon Glue:** Un servicio de integración de datos totalmente administrado que permite descubrir, preparar y mover datos.



Apache Kafka es una **plataforma de mensajería distribuida** diseñada para manejar el procesamiento de flujos de datos en tiempo real. Kafka permite publicar, suscribirse, almacenar y procesar transmisiones de datos de manera eficiente y escalable. A diferencia de los sistemas de mensajería tradicionales, que son generalmente más simples y menos orientados a manejar grandes volúmenes de datos, Kafka se destaca por su alta **durabilidad, escalabilidad horizontal y bajo acoplamiento** entre los productores y consumidores de datos.

En términos simples, Kafka actúa como un "intermediario" o "buffer" que facilita la transmisión de datos (mensajes) entre sistemas que producen datos (producers) y sistemas que consumen o procesan esos datos (consumers).

Apache Kafka fue creado en **2011** por ingenieros de LinkedIn (Jay Kreps, Neha Narkhede y Jun Rao) para resolver un problema particular: la creciente necesidad de procesar enormes cantidades de datos generados en tiempo real dentro de la red social profesional. LinkedIn se enfrentaba a desafíos en la administración y procesamiento de flujos de eventos y datos de diversas fuentes, como actividad de usuarios, métricas operativas y registros de aplicaciones.

Antes de Kafka, LinkedIn, al igual que muchas otras empresas, usaba una combinación de bases de datos y herramientas tradicionales para manejar sus datos, pero estas soluciones no eran suficientemente rápidas ni eficientes para manejar el volumen creciente de información. Se necesitaba una solución que pudiera:

- **Escalar** de forma horizontal en cientos de servidores.
- Ofrecer **alta tolerancia a fallos**.
- Manejar grandes volúmenes de datos en tiempo real sin comprometer el rendimiento.
- Proporcionar una **baja latencia** en la entrega de mensajes.

Para resolver estos problemas, Apache Kafka fue diseñado con los principios de:

1.Almacenamiento distribuido: Los datos se distribuyen en múltiples servidores o "brokers", y se pueden replicar para aumentar la durabilidad.

2.Procesamiento en tiempo real: Kafka está optimizado para manejar y procesar datos a medida que se generan, en lugar de ser un sistema de procesamiento por lotes.

3.Alta escalabilidad: Kafka puede manejar millones de mensajes por segundo con una infraestructura adecuada, lo que lo convierte en una herramienta adecuada para sistemas de gran escala.

4.Persistencia: A diferencia de muchos sistemas de mensajería, Kafka almacena mensajes durante períodos de tiempo configurables, lo que permite a los consumidores leer mensajes antiguos según sea necesario.



Apache Kafka destaca por sus funcionalidades avanzadas, que lo convierten en una plataforma ideal para manejar grandes volúmenes de datos en tiempo real. A continuación, se detallan las principales capacidades de Kafka y cómo estas permiten que se utilice en una amplia variedad de casos de uso en la industria.

1. Sistema de Mensajería Distribuido

La funcionalidad central de Kafka es su capacidad para actuar como un **sistema de mensajería distribuido**. En este modelo, Kafka facilita la comunicación entre diferentes sistemas al permitir que las aplicaciones intercambien mensajes (datos) de forma asíncrona. Los mensajes se agrupan en **topics**, que son categorías o feeds específicos a los que los **producers** (quienes generan los mensajes) envían datos y los **consumers** (quienes leen los mensajes) recuperan datos.

Ventajas de este modelo:

- **Desacoplamiento de sistemas:** Los productores y consumidores de mensajes no necesitan interactuar directamente entre sí, lo que permite que los sistemas sean más modulares y escalables.
- **Procesamiento asíncrono:** Los sistemas no necesitan procesar los datos al mismo tiempo. Los mensajes pueden ser almacenados temporalmente en Kafka hasta que los consumidores estén listos para procesarlos.

2. Alta Escalabilidad

Kafka fue diseñado desde el principio para manejar grandes volúmenes de datos distribuidos en clusters de servidores. Las principales características que permiten la escalabilidad de Kafka incluyen:

- **Particionamiento de Topics:** Cada topic en Kafka se divide en varias particiones, y los mensajes se distribuyen entre estas particiones. Esto permite que el procesamiento de mensajes sea distribuido en múltiples nodos (brokers), lo que mejora el rendimiento y facilita la escalabilidad horizontal.
- **Balanceo de carga:** Kafka balancea la carga de mensajes de manera automática. Cuando el número de consumidores aumenta, Kafka distribuye los mensajes entre ellos, optimizando el uso de los recursos disponibles.

Por ejemplo, Kafka puede procesar millones de mensajes por segundo en una infraestructura adecuada, siendo ideal para sistemas de alta demanda y grandes volúmenes de datos, como monitoreo de eventos o procesamiento de logs en tiempo real.

3. Durabilidad y Persistencia

Uno de los puntos más fuertes de Kafka es su capacidad para **almacenar datos de manera persistente**. A diferencia de los sistemas tradicionales de mensajería (como RabbitMQ o ActiveMQ), que normalmente eliminan los mensajes una vez que son consumidos, Kafka permite configurar **políticas de retención** para que los datos permanezcan almacenados durante un periodo de tiempo determinado, incluso si ya han sido leídos por los consumidores.

- **Log de almacenamiento:** Kafka organiza los mensajes en logs que se almacenan en disco de manera persistente. Esto asegura que los datos puedan recuperarse en caso de fallos o reinicios.
- **Política de retención:** Los mensajes pueden configurarse para que permanezcan en Kafka durante un periodo definido (por ejemplo, días, semanas) o hasta que se alcance un límite de almacenamiento. Esto lo convierte en una herramienta poderosa para auditorías o análisis retrospectivo de eventos.
- **Réplicas:** Los datos en Kafka se replican en múltiples nodos para garantizar que, incluso si un servidor falla, los datos no se pierdan. Esto mejora la durabilidad y garantiza la alta disponibilidad.

4. Bajo Acoplamiento y Alta Disponibilidad

Kafka está diseñado para ser **altamente disponible** y tolerante a fallos. Gracias a su arquitectura distribuida, es posible que un nodo (broker) del sistema falle sin afectar el procesamiento general de los mensajes, ya que las réplicas aseguran que los datos se encuentren disponibles en otros nodos.

- **Tolerancia a fallos:** Kafka asegura la disponibilidad de los mensajes mediante el uso de réplicas. Si un nodo falla, otro nodo puede continuar procesando los mensajes sin pérdida de datos.
- **Rebalanceo automático:** Cuando un nodo o partición falla, Kafka redistribuye los datos de manera automática para continuar con la operación normal sin intervención manual.

5. Procesamiento de Flujos en Tiempo Real

Kafka permite procesar **flujos de datos en tiempo real**, lo que significa que las aplicaciones pueden procesar datos en el momento en que son producidos, en lugar de hacerlo en intervalos de tiempo o lotes. Esto es fundamental para muchas aplicaciones críticas que requieren decisiones y respuestas rápidas, como:

- **Monitorización de eventos:** Aplicaciones que requieren la detección de anomalías o patrones en el tráfico de red o en sistemas de seguridad.
- **Análisis en tiempo real:** Por ejemplo, en comercio electrónico para analizar el comportamiento de los usuarios en tiempo real y ofrecer recomendaciones personalizadas.

Kafka facilita el procesamiento de flujos en tiempo real a través de herramientas como **Kafka Streams** y **ksqlDB**, que permiten realizar análisis y transformaciones de datos directamente dentro del ecosistema de Kafka.

6. Integración con Diversos Sistemas

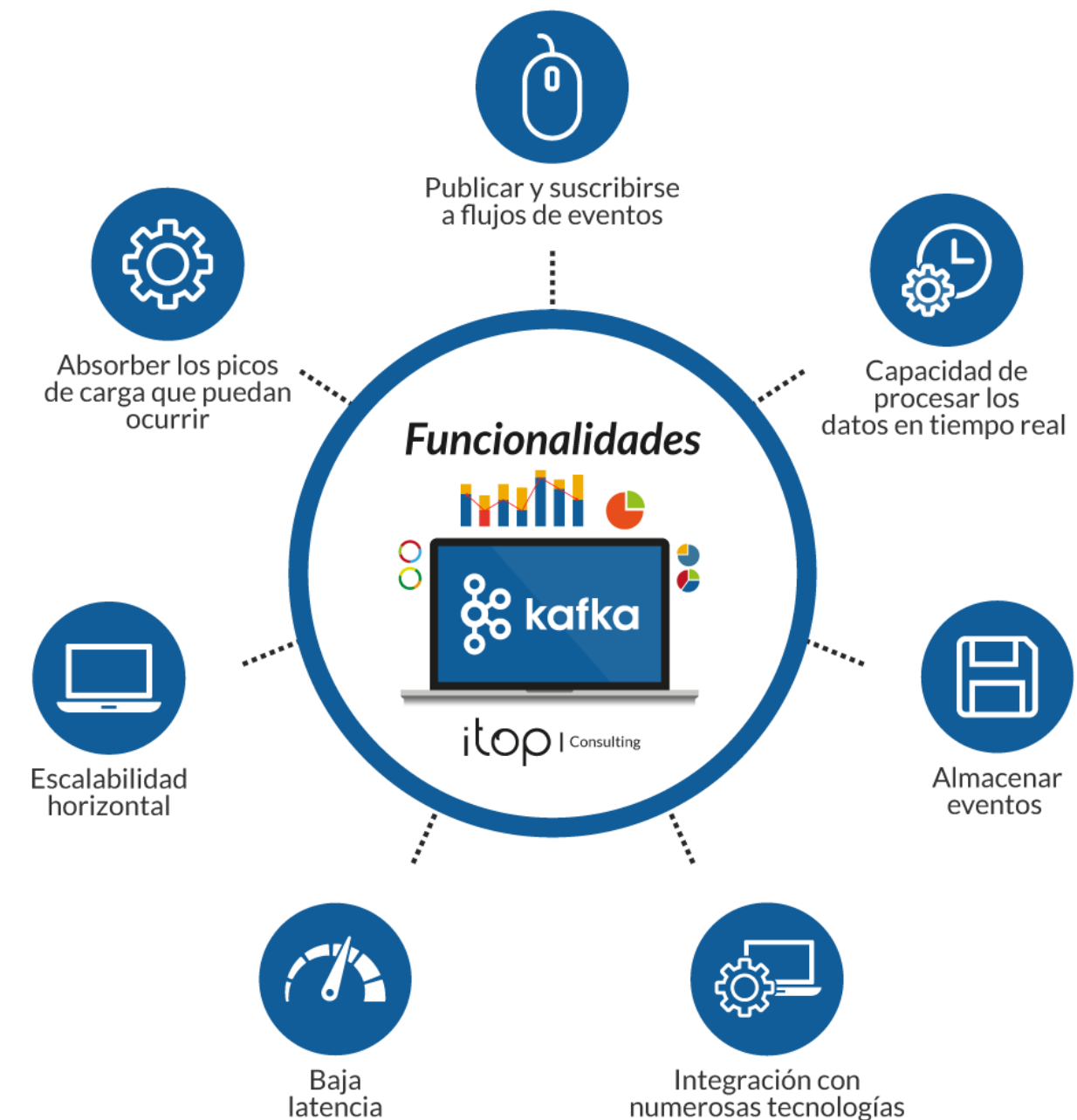
Kafka se integra fácilmente con una amplia gama de sistemas externos, lo que lo convierte en una plataforma central en arquitecturas modernas de datos. A través de **Kafka Connect**, se puede conectar Kafka con:

- **Bases de datos:** Kafka puede recibir o enviar datos a bases de datos relacionales (MySQL, PostgreSQL) y no relacionales (MongoDB, Cassandra).
- **Sistemas de análisis:** Kafka puede enviar datos a sistemas de procesamiento de big data como Hadoop, Spark o Flink.
- **Almacenamiento en la nube:** Kafka también puede integrarse con servicios de almacenamiento en la nube como AWS S3, Google Cloud Storage y Azure Blob Storage.

7. Procesamiento de Mensajes a Gran Escala con Consumer Groups

Kafka permite que múltiples consumidores lean mensajes de un mismo topic al dividir el trabajo entre ellos, a través de **Consumer Groups**. Esto permite:

- **Escalabilidad del consumo:** Los mensajes de un topic se distribuyen entre los consumidores del grupo, permitiendo que cada consumidor procese solo una parte de los mensajes. Esto facilita la distribución eficiente de la carga de procesamiento.
- **Procesamiento paralelo:** Varios consumidores pueden procesar mensajes en paralelo, lo que es esencial para manejar grandes volúmenes de datos de manera eficiente.



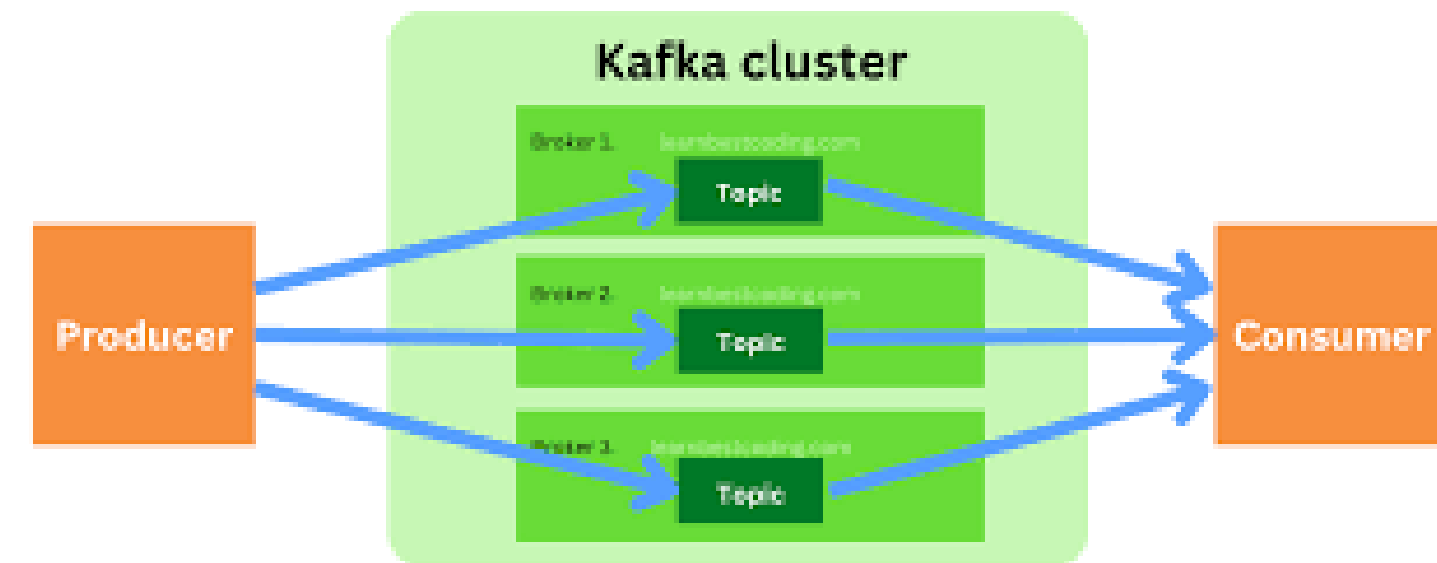
Funcionalidades Principales de Apache Kafka

La arquitectura de Apache Kafka está diseñada para gestionar flujos masivos de datos de manera eficiente, escalable y tolerante a fallos. Comprender los componentes y cómo interactúan entre sí es clave para aprovechar todo el potencial de Kafka. A continuación se detalla su estructura y los elementos principales.

1. Cluster de Kafka

En Kafka, los datos se distribuyen en un **cluster** formado por múltiples servidores, conocidos como **brokers**. El cluster de Kafka es una red de nodos que trabajan en conjunto para almacenar y distribuir mensajes. La arquitectura del cluster permite:

- **Alta disponibilidad:** Si uno de los brokers falla, otros brokers pueden asumir su rol para garantizar la continuidad del servicio.
- **Escalabilidad horizontal:** A medida que aumenta el volumen de datos o el número de consumidores, se pueden añadir más brokers al cluster sin interrupciones.



Componentes Principales de Apache Kafka

Los principales actores en el sistema Kafka son:

a. Producers (Productores)

Los **producers** son las aplicaciones o servicios que **generan y envían mensajes** a Kafka. Los productores publican los mensajes en **topics** específicos. Algunas características clave de los producers:

Elección de particiones: Un productor puede decidir en qué partición de un topic específico almacenar un mensaje. Puede hacerlo aleatoriamente o utilizando una clave para garantizar que los mensajes relacionados vayan a la misma partición.

Confirmación de envío (acks): Los productores pueden configurar cómo Kafka debe confirmar la recepción de los mensajes. Por ejemplo, pueden esperar confirmación de todos los brokers o solo de uno para equilibrar rendimiento y durabilidad.

b. Consumers (Consumidores)

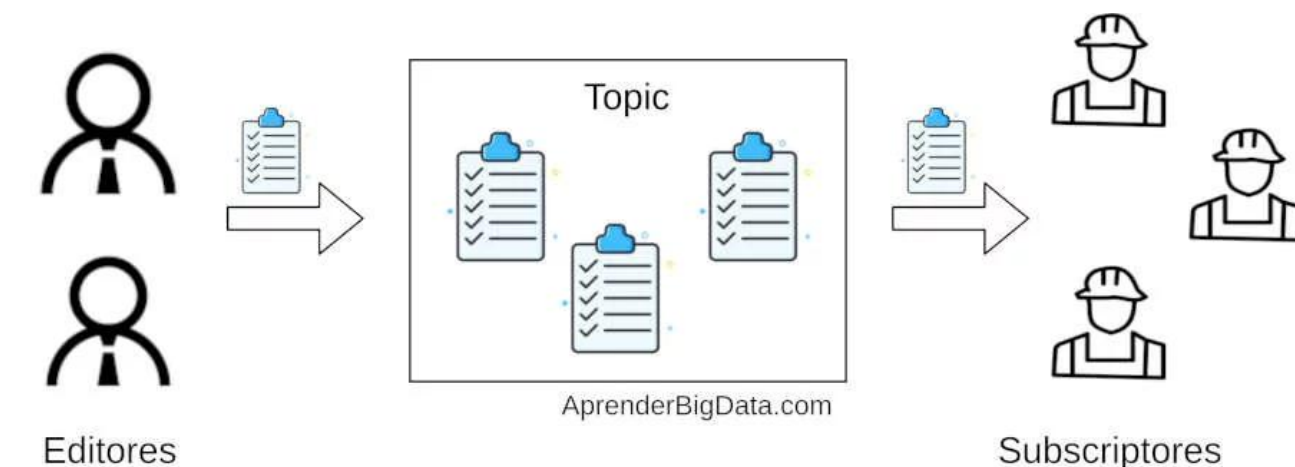
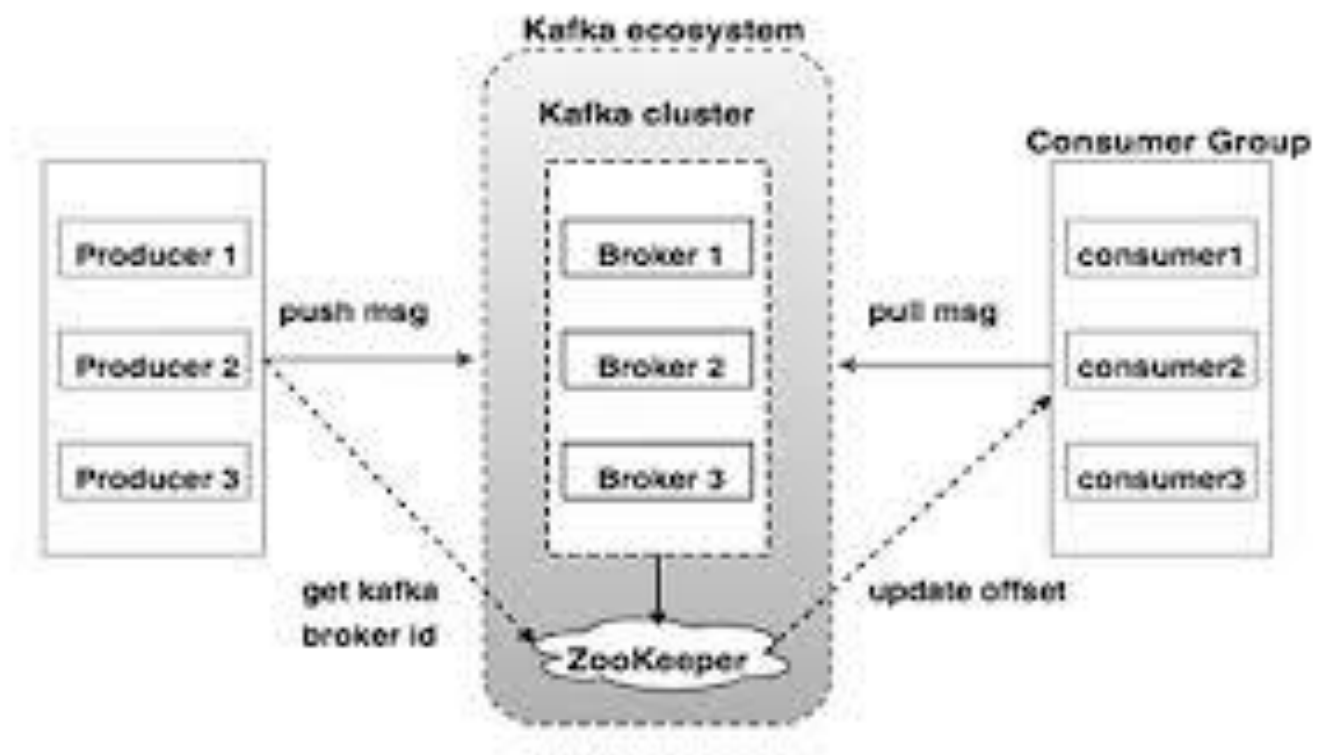
Los **consumers** son aplicaciones que **leen y procesan los mensajes** de Kafka. Se suscriben a uno o varios **topics** y consumen los mensajes en el orden en que fueron publicados. Las características principales son:

- **Consumer Groups:** Los consumidores pueden agruparse en **grupos de consumidores**. Cada grupo de consumidores puede leer mensajes de manera independiente. Kafka distribuye automáticamente las particiones de un topic entre los consumidores dentro de un mismo grupo, lo que permite un procesamiento paralelo.
- **Offsets:** Los consumidores mantienen un registro llamado **offset** que marca el último mensaje leído en una partición. Esto permite que los consumidores reanuden el procesamiento desde donde lo dejaron en caso de un fallo.

c. Brokers (Nodos del Cluster)

Los **brokers** son los servidores que forman el **cluster de Kafka**. Cada broker se encarga de:

- **Almacenar mensajes:** Los brokers almacenan los mensajes de las particiones de los topics. Cada mensaje tiene un **offset** único dentro de la partición que permite a los consumidores leer los mensajes en orden.
- **Gestión de particiones:** Un broker puede ser el líder de una partición, lo que significa que coordina las operaciones de lectura y escritura de esa partición. Los otros brokers mantendrán réplicas de los datos para asegurar la alta disponibilidad.



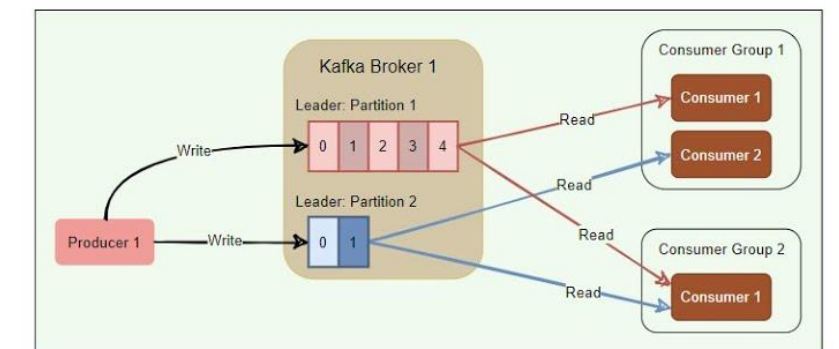
d. Zookeeper (o KRaft en nuevas versiones)

Zookeeper es un sistema de coordinación distribuida que Kafka utilizaba tradicionalmente para tareas de administración del cluster, como:

- **Almacenamiento de metadatos** sobre brokers, topics, particiones y consumidores.
- **Elección del broker líder** para cada partición.
- **Monitoreo de la salud del cluster.**

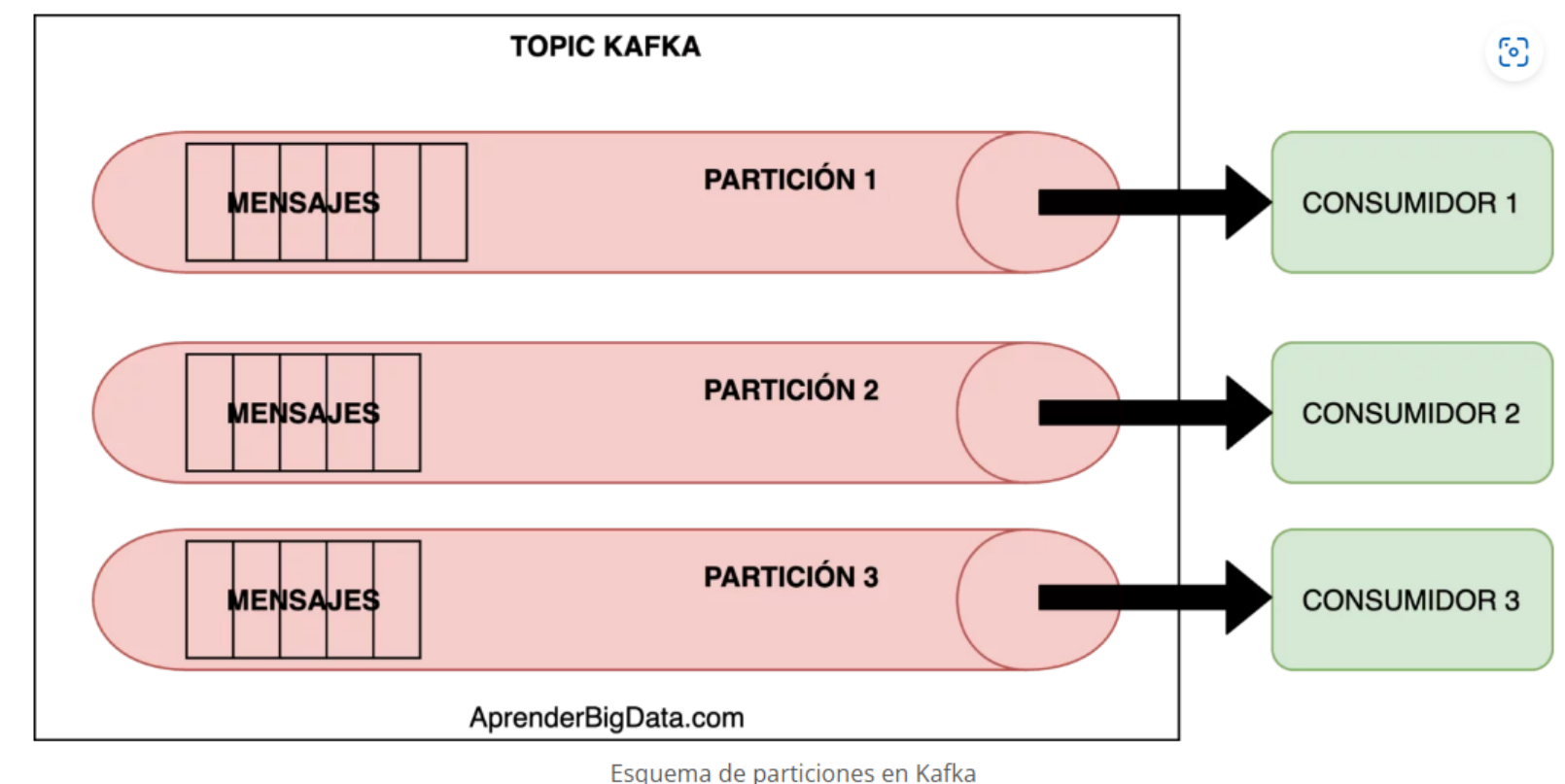
Sin embargo, en las versiones más recientes de Kafka, se ha empezado a reemplazar Zookeeper con una nueva arquitectura llamada **KRaft**, que es más eficiente y elimina la dependencia de Zookeeper para la coordinación del cluster.

Kafka:- Role of Zookeeper



Un **topic** en Kafka es una categoría o feed específico al que los productores envían mensajes y los consumidores se suscriben. Un topic se subdivide en múltiples **particiones** para mejorar la escalabilidad y permitir la paralelización del procesamiento. Cada partición actúa como un log de datos ordenado:

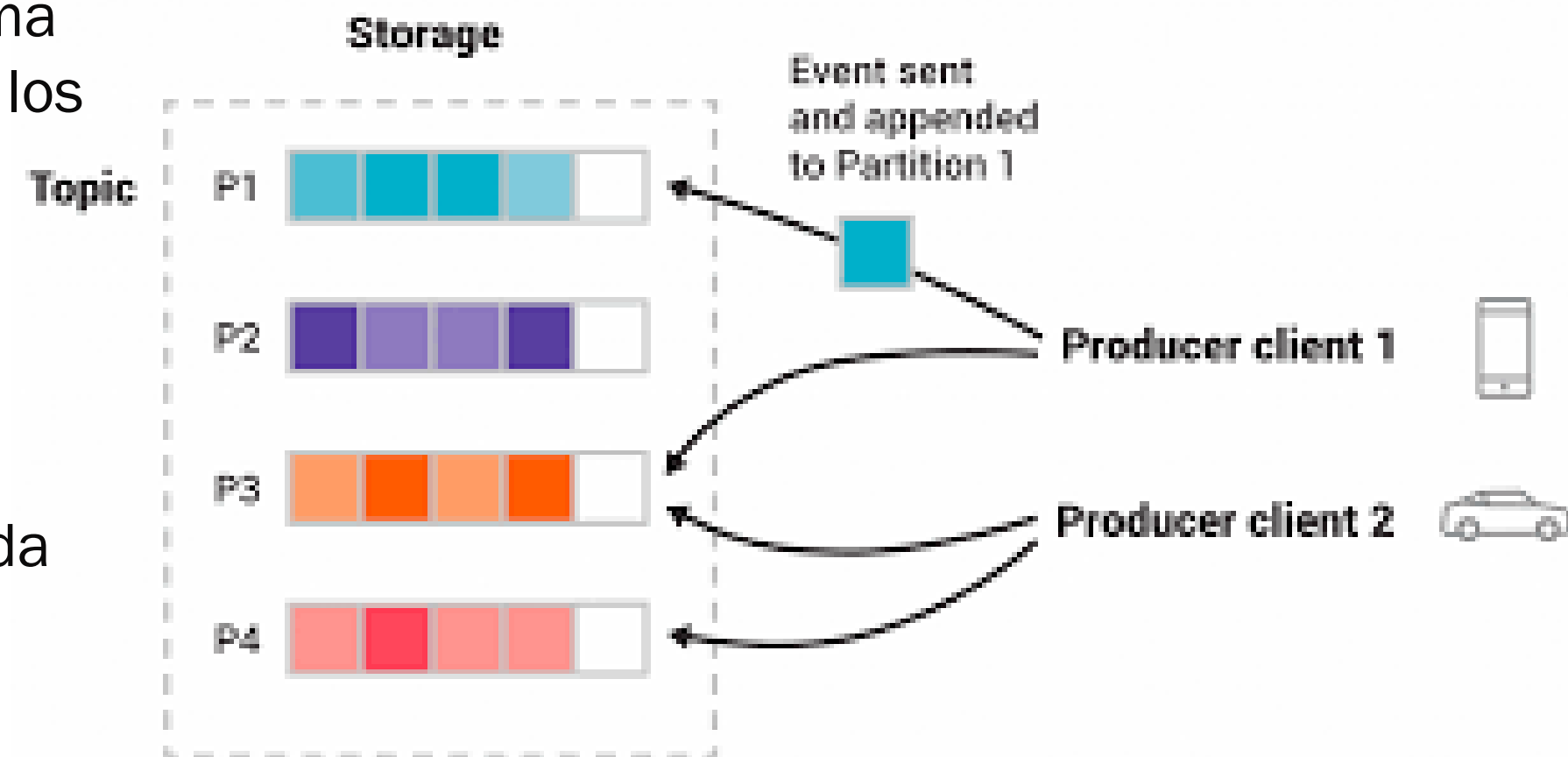
- **Particionamiento:** Cuando un productor envía un mensaje a un topic, Kafka distribuye los mensajes en varias particiones. Este esquema de particionamiento permite que diferentes consumidores lean las particiones en paralelo, mejorando el rendimiento.
- **Particiones como logs ordenados:** Los mensajes dentro de una partición son **ordenados secuencialmente** y cada mensaje tiene un **offset**, que es un identificador único de su posición dentro de la partición. Los consumidores pueden leer los mensajes en el orden en que se almacenaron, y cada consumidor mantiene un registro de hasta dónde ha leído en cada partición utilizando el offset.
- **Réplicas:** Para garantizar la **tolerancia a fallos**, Kafka permite crear múltiples réplicas de una partición en otros brokers. Un broker actúa como el **líder** de una partición, mientras que otros brokers mantienen réplicas de respaldo. Si el broker líder falla, uno de los brokers con réplicas toma el control.



4. Particiones y Réplicas

Cada **topic** en Kafka se divide en particiones, y cada partición se almacena en un broker del cluster. Kafka implementa un sistema de **réplicas** para garantizar la durabilidad y la disponibilidad de los datos. Las particiones y réplicas se organizan de la siguiente manera:

- **Partición líder y réplicas:** Dentro del cluster, cada partición tiene un **líder** y varias **réplicas**. El líder es responsable de manejar todas las operaciones de lectura y escritura de la partición. Las réplicas solo mantienen una copia sincronizada de los datos.
- **Failover automático:** Si el broker líder de una partición falla, uno de los brokers con réplicas asume el rol de líder de forma automática, asegurando la continuidad del servicio sin pérdida de datos.

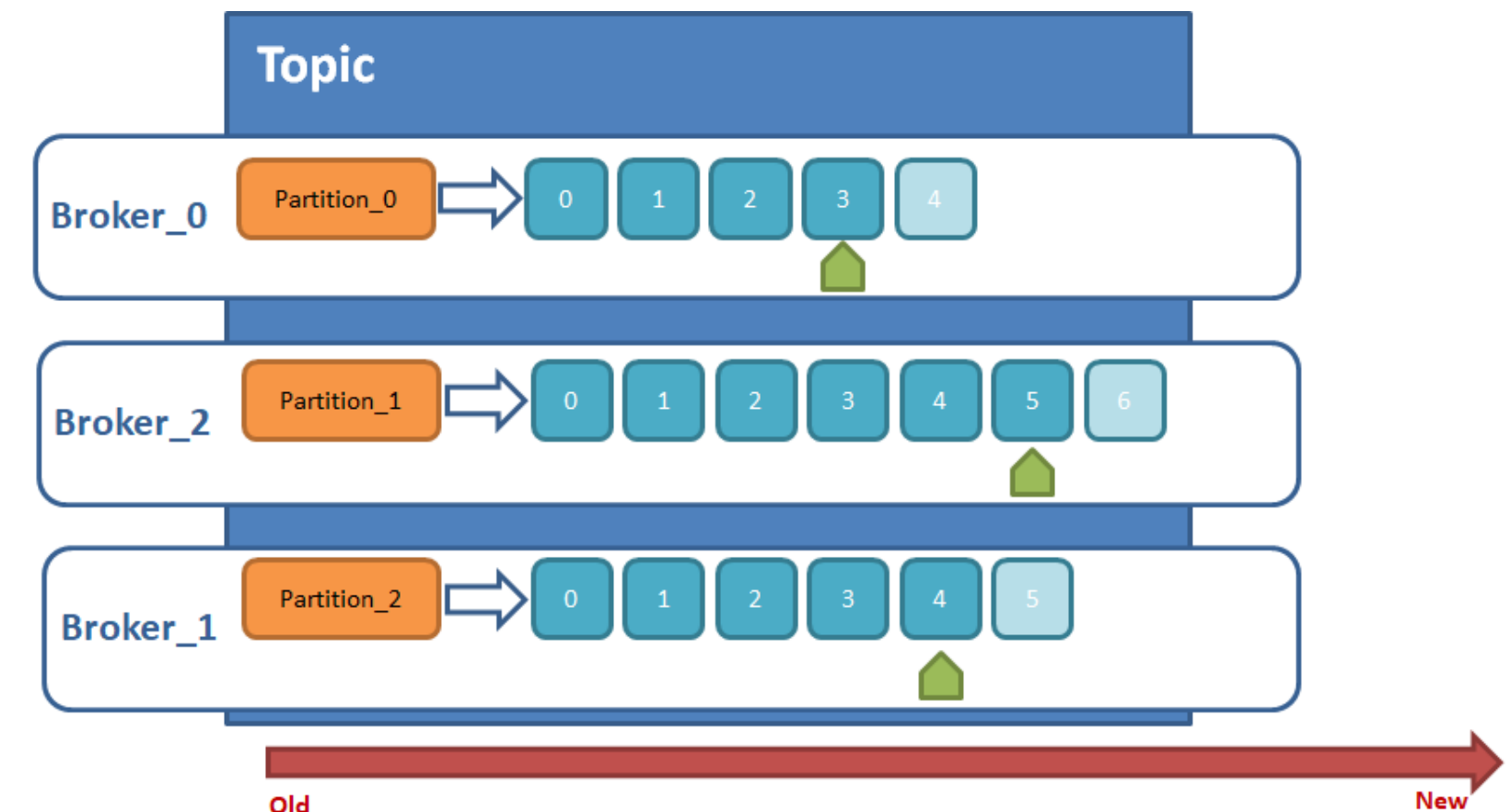


5. Retención de Datos

Una de las funcionalidades más poderosas de Kafka es su capacidad de almacenar mensajes durante un periodo determinado a través de políticas de **retención de datos**. A diferencia de otros sistemas de mensajería que eliminan los mensajes una vez que son consumidos, Kafka permite configurar cuánto tiempo se almacenan los mensajes en disco, independientemente de si los consumidores ya los han leído.

- **Retención basada en tiempo:** Los mensajes pueden configurarse para permanecer en Kafka durante un tiempo definido, por ejemplo, 7 días.
- **Retención basada en tamaño:** También es posible configurar Kafka para que mantenga los mensajes hasta que el almacenamiento de una partición alcance un tamaño determinado.

Esta capacidad de retención hace que Kafka sea ideal no solo para la mensajería en tiempo real, sino también para análisis históricos y auditorías de datos, ya que los mensajes antiguos siguen estando disponibles durante un tiempo.



- Un topic se divide en particiones
- Cada partición se encuentra en un bróker actuando como leader -> Distribución
- Cada partición leader maneja todas las peticiones de lectura o escritura sobre ella
- Si existe factor de replicación, la partición leader tendrá "copias" en el resto de brokers -> particiones followers o replicas
- Si la partición leader falla una de las particiones followers pasará a ser leader

INTRODUCCIÓN APACHE KAFKA

 Aprender BIG DATA
AprenderBigData.com

Apache Kafka para principiantes

por Andrés Gómez Ferrer

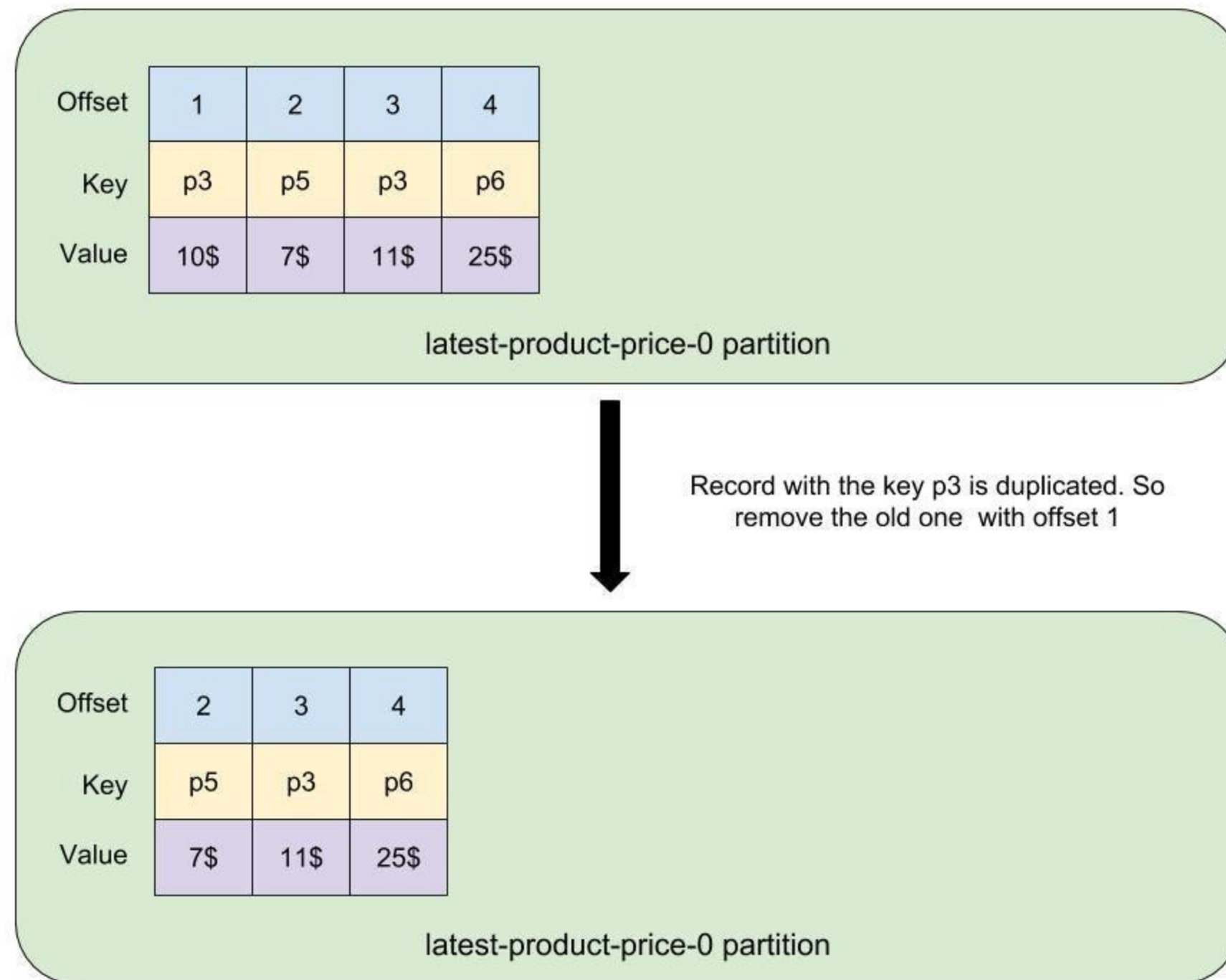
 **KEEPCODING**
Tech School



6. Logs de Mensajes

Kafka organiza los mensajes en un formato de **log** distribuido. Cada partición de un topic es, en esencia, un **log inmutable** e que los mensajes se añaden de forma secuencial. Este sistema de logs es crucial para:

- **Garantizar la durabilidad de los mensajes:** Los mensajes se almacenan en disco y pueden ser leídos varias veces por diferentes consumidores.
- **Lectura secuencial:** Al procesar los mensajes en orden secuencial, Kafka puede proporcionar un alto rendimiento en las operaciones de lectura y escritura.



Se trata de una plataforma de streaming basada en Apache Kafka cuyo objetivo es **ayudar a los diferentes roles en la organización en los proyectos de intercambio de información en tiempo real.**

Para ello, desde un punto de vista de arquitectura tecnológica, mejora las capacidades nativas base de Apache Kafka en las siguientes perspectivas:

- Capacidades de Streaming Enterprise
- Conectividad y Gobierno del Dato
- Procesamiento Avanzado
- Monitorización y Gestión
- Seguridad
- Arquitecturas MultiSite y Apoyo en la evolución hacia el Cloud



CONFLUENT

Es capaz de:

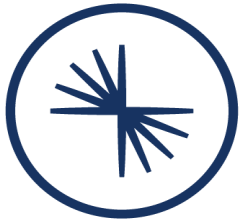
- Integrar fuentes productoras y consumidoras de eventos de diferente naturaleza
- Gestionar las capacidades Real Time para transporte de altos volúmenes de datos (Megabytes por segundo)
- Garantizar la resiliencia, escalabilidad y secuencialidad en la transferencia de eventos.
- Colaborar con el resto de los componentes de arquitectura para la gestión de los metadatos, la distribución de carga dentro del clúster.
- Mediante Confluent Server se consiguen maximizar las capacidades de Streaming



Fuente: Confluent

Explore nuestros casos de uso más populares

Detección del fraude	Microservicios event-driven	Pipelines de bases de datos	Integración del mainframe	Pipelines de almacenes de datos	Integración de mensajería
Optimización de SIEM	Análisis de IoT en tiempo real	Reabastecimiento en tiempo real	Redes autónomas	IA generativa	Gestión de riesgos
Análisis en tiempo real	Integración de Security Lake	Análisis predictivo	Inventario en tiempo real	Análisis de vendedores en tiempo real	Pagos en tiempo real



CONFLUENT

Soluciones y casos de uso de Confluent | ES

Gracias